UNIVERSITY
OF SUSSEX

Masters Thesis

Placeholder title

A thesis submitted in fulfilment of the requirements
for the degree of Masters of Research

in the

School of Engineering and Informatics

Author:
Henry Williams

Supervisor:
Jeff Mitchell

August 2025

Chapter 1

Introduction

Word Count: 1000

- What is malware?
- Why is detecting it important?
- How do we detect it?
- How does it evade detection?
- What are we doing?

Chapter 2

Literature Review

Word Count: 2000

- Malware
  - Definition
  - Forms of malicious software
  - Common features (obfuscation, anti-debugging, etc)
- Machine Learning (?)
- Existing Approaches
  - Differences between static/hybrid/dynamic approaches
  - Compare differences between common static, and dynamic features
  - Compare differences in performance between static and dynamic approaches
  - Introduce approach taken by commerical solutions
  - Conclusion: *"Despite being more accurate at detecting malware, dynamic approaches are limited by the need to execute suspicious programs in an isolated environment, leading to increased resource requirements. However, static approaches are more likely to be vulnerable to obfuscation."*

Chapter 3

Methodology

Word Count: 3000

- What: *"Do static malware detectors learn to detect malware based on the presence of obfuscation alone?"*
- Why?
  - Develop a better understanding of the barriers to the success of static malware detectors
  - Establish whether NLP techniques are well suited to modelling executable programs
- How?
  - Dataset & analysis
  - Baseline approaches
  - Opcode sequence classifier based on RoBERTa
  - Image classifier based on efficientnet
  - Secondary dataset
  - UPX obfuscation
  - Experiment set-up

Chapter 4

Results

Word Count: 3000

- Baseline results
- Opcode model
  - Is pretraining useful?
  - How best to reduce sequence-level classification results to a file-level result?
- Image model
  - Is increased classification accuracy a result of the difference between the distribution between the size of malicious and benign programs?
- Obfuscation Experiment
  - Opcode Model
    - Baseline file-level classification accuracy on secondary dataset
    - When both classes of programs, or soley malicious programs are obfuscated, performance increases
    - When only benign programs are obfuscated, accuracy doesn't increase
    - Less pronounced in sequence level tasks
    - Certainty (D_{KL}) of classification increases as more programs are obfuscated
  - TODO: Image model

Chapter 5

Discussion

Word Count: 4000

- Are dynamic methods vulnerable to advanced obfuscation techniques?
  - It's possible to hide API calls from standard API call hooking techniques
  - Other redundant API calls may be made to obfuscate call sequences
  - Advanced malware (i.e. Rootkits) are able to stealthily perform malicious actions on the host
- Are opcode sequences suitable features for NLP based malware detection?
  - Full disassembly may offer provide better information for transformer architectures
  - Was not tested due to time/compute limitations
  - Byte-level modelling may work, but the presence of polymorphic malware and encrypted payloads may be challenging to model accurately
- Is executable packing (UPX) a good proxy for the obfuscation methods seen in the wild?
  - What other obfuscation techniques and solutions could work for this?
- Could diffusion modelling enable machine de-obfuscation?
  - Obfuscation is analgous to noise in a signal
  - Diffusion models are well suited to denoising of data
  - Might be able to de-obfuscate programs to generate large scale
- Data limitations
  - Lack of large-scale, benign executable datasets
  - High presence of obfuscation in real-world malware samples
  - Unobfuscated malware samples are not representative of malware seen in the wild as they are often "toy" projects, and lack advanced capabilities
  - Unobfuscated, advanced malware are hard to find
- Conclusion